Parte	Tarea	Estado
A · L1	Tabla Matricula (clave compuesta + fecha del servidor)	Hecho
B · L2	Inscribirme (USER, auth.uid, idempotente)	Hecho
B · L3	MisInscripciones (solo las del usuario que llama)	Hecho
B · L5	DarmeDeBaja (opcional)	Hecho
C · L4	Auditoría con npm run atacar y corrección de ListAlumnos	Hecho
D · L6/L7	CSP con orígenes exactos y deploy de SQL Connect	Hecho

Parte A · Modelado relacional (L1)

Matricula modela la relación muchos-a-muchos entre Alumno y Curso. La clave primaria es compuesta por **alumno** y **curso**, y la **fecha** la completa el servidor con request.time. Los campos alumno: Alumno! y curso: Curso! generan las claves foráneas alumnoId y cursoId.

```
type Curso @table {
  titulo: String!
  nivel: String!        # "inicial" | "intermedio" | "avanzado"
  horas: Int!
  descripcion: String!
}

type Alumno @table {
  id: String! @default(expr: "auth.uid")
  nombre: String!
  email: String!
}

type Matricula @table(key: ["alumno", "curso"]) {
  alumno: Alumno!
  curso: Curso!
  fecha: Timestamp! @default(expr: "request.time")
}
```

Parte B · Operaciones con autorización (L2, L3, L5)

Ninguna operación recibe el id del alumno: siempre sale del token con auth.uid. Inscribirme usa **upsert** sobre la clave compuesta, por eso inscribirse dos veces no falla ni duplica (con insert, la segunda llamada violaría la PK).

```
mutation UpsertAlumno($nombre: String!) @auth(level: USER) {
  alumno_upsert(data: {
    id_expr: "auth.uid"
    nombre: $nombre
    email_expr: "auth.token.email"
  })
}

mutation Inscribirme($cursoId: UUID!) @auth(level: USER) {
  matricula_upsert(data: {
```

```
    alumnoId_expr: "auth.uid"
    cursoId: $cursoId
  })
}

mutation DarmeDeBaja($cursoId: UUID!) @auth(level: USER) {
  matricula_delete(key: {
    alumnoId_expr: "auth.uid"
    cursoId: $cursoId
  })
}


query ListCursos
  @auth(level: PUBLIC, insecureReason: "Catálogo público de cursos: no contiene datos personales.") {
  cursos(orderBy: [{ titulo: ASC }]) {
    id
    titulo
    nivel
    horas
    descripcion
  }
}

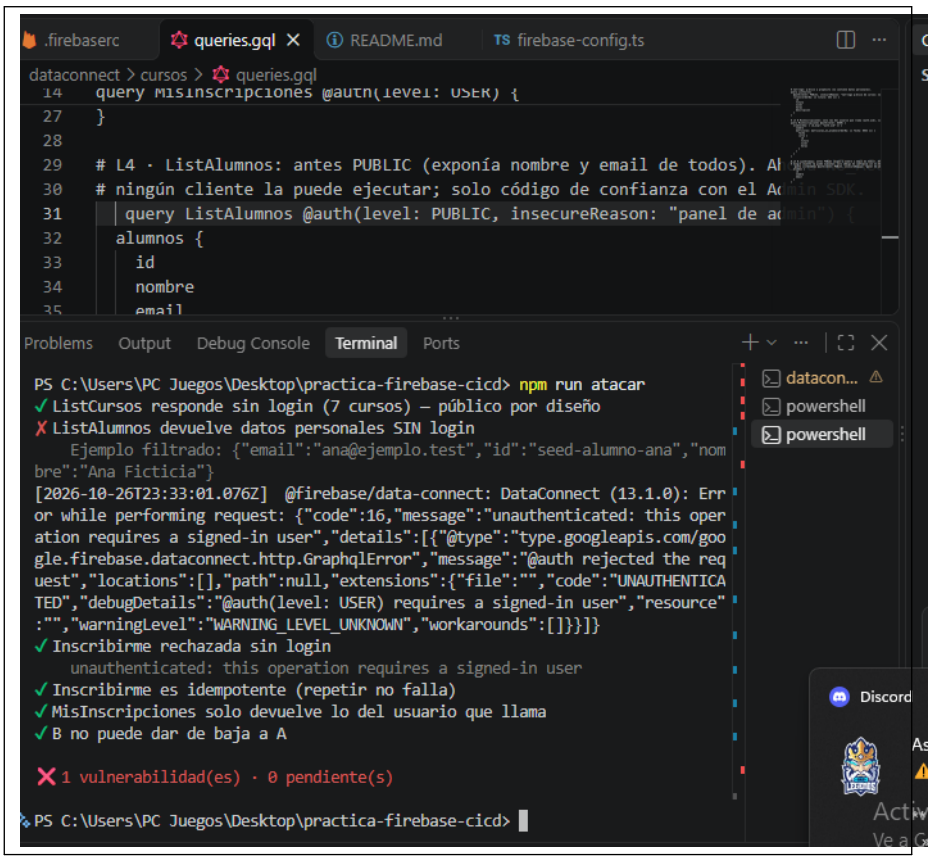
query MisInscripciones @auth(level: USER) {
  alumno(key: { id_expr: "auth.uid" }) {
    nombre
    matriculas: matriculas_on_alumno(orderBy: [{ fecha: DESC }]) {
      fecha
      curso {
        id
        titulo
        nivel
        horas
      }
    }
  }
}

query ListAlumnos @auth(level: NO_ACCESS) {
  alumnos {
    id
    nombre
    email
  }
}
```

Parte C · Auditoría de seguridad (L4)

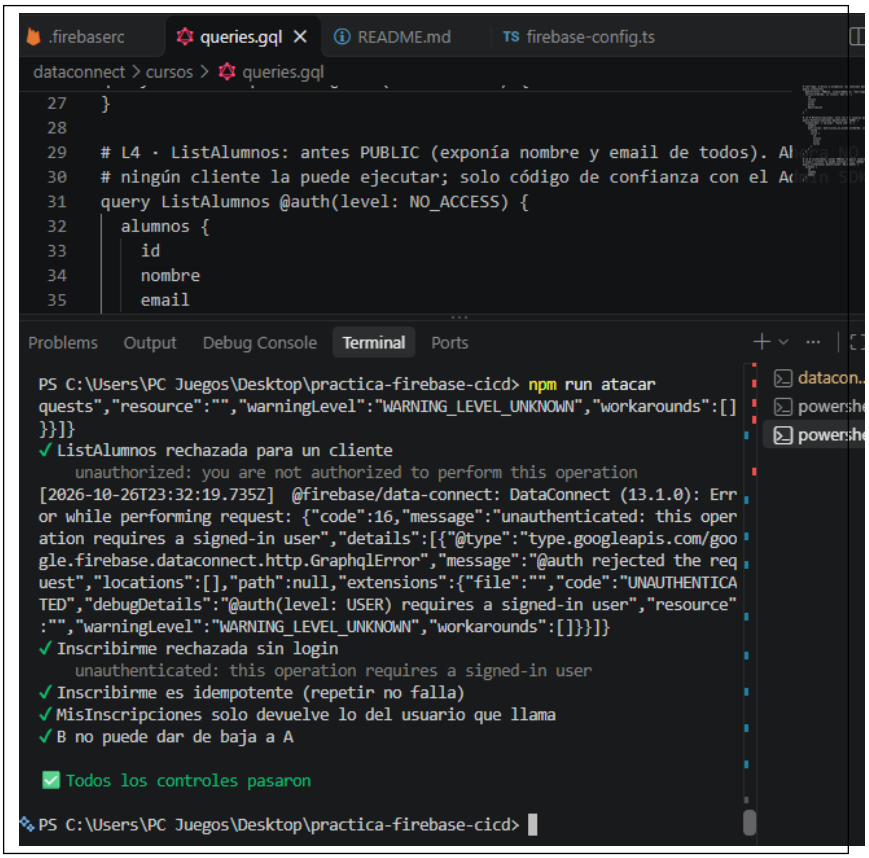
npm run atacar llama a las operaciones con la configuración pública de la app, como lo haría un atacante. La operación heredada **ListAlumnos** estaba en PUBLIC: cualquiera, sin login, podía leer nombre y email de todos los alumnos (datos personales). Se corrigió a NO_ACCESS.

Antes de corregir: ListAlumnos en PUBLIC



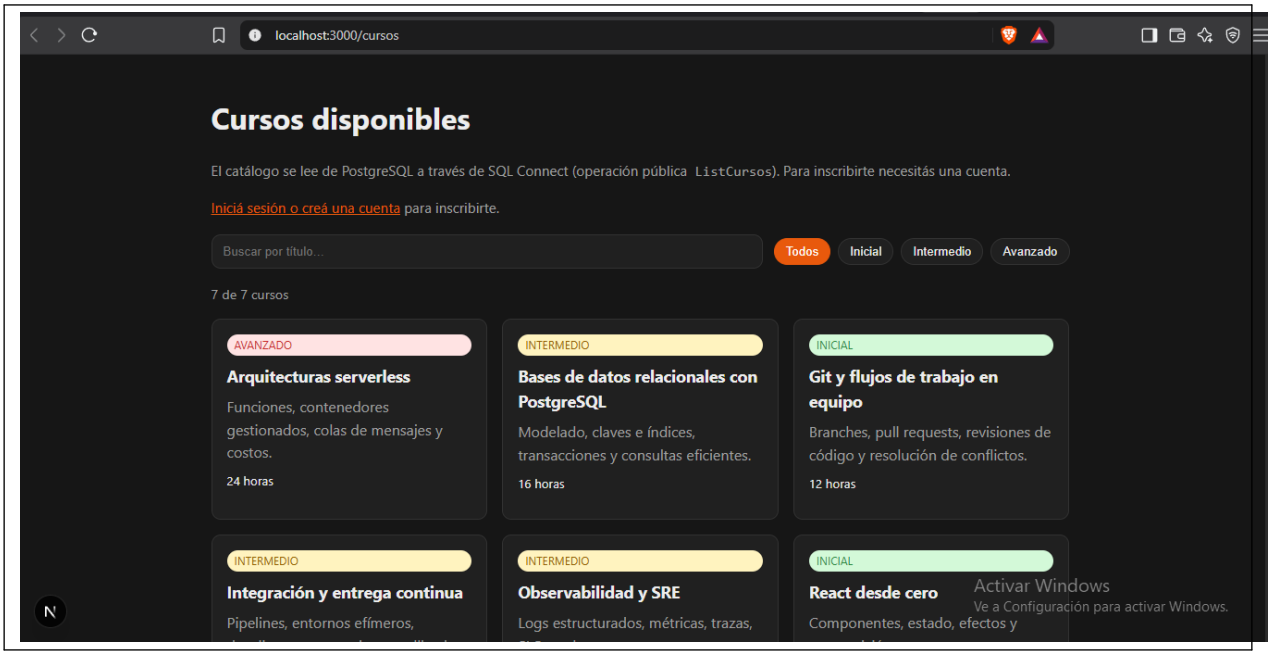
Captura 1 · La auditoría detecta 1 vulnerabilidad: ListAlumnos devuelve datos personales sin login.

Después de corregir: ListAlumnos en NO_ACCESS

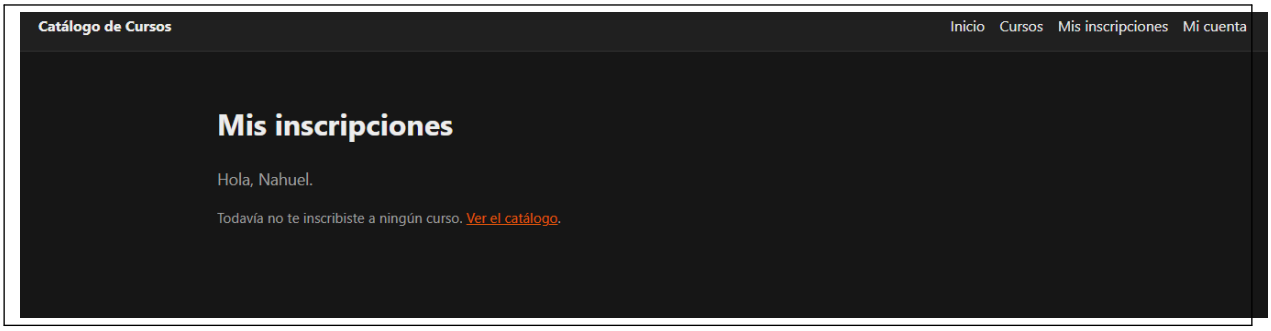


Captura 2 · Todos los controles pasan: ListAlumnos rechazada, Inscribirme exige login y es idempotente, y hay aislamiento de lectura y escritura entre usuarios.

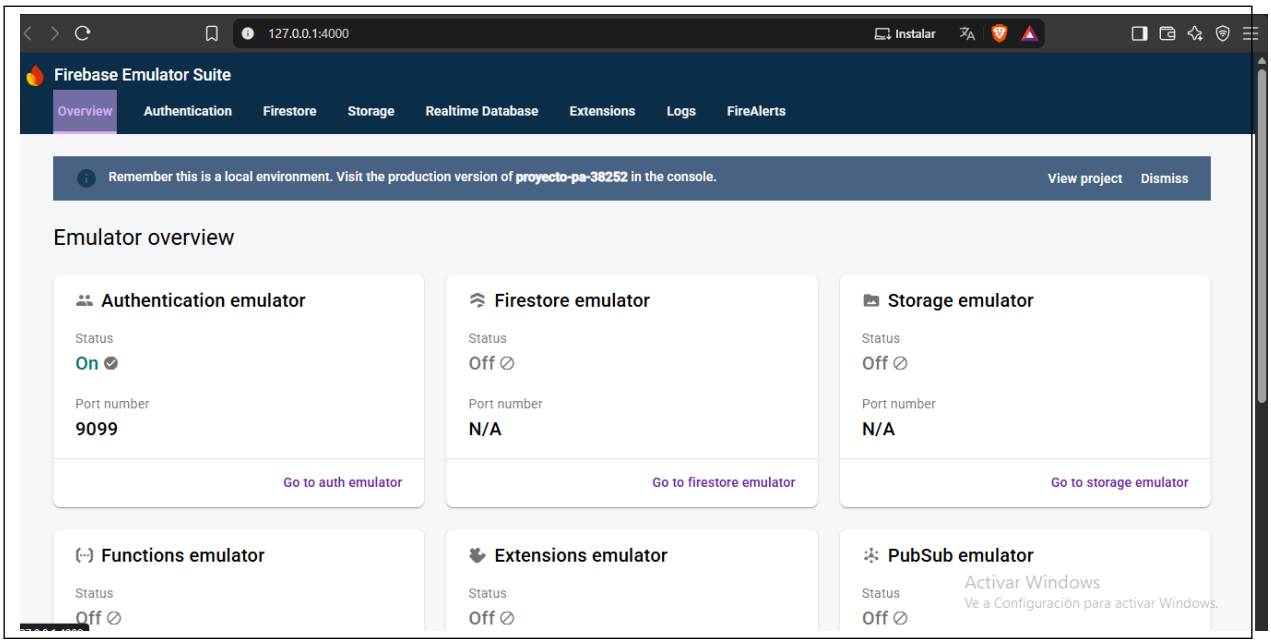
Pruebas en la aplicación



Captura 3 · Catálogo en localhost:3000/cursos: los 7 cursos se leen de PostgreSQL con la operación pública ListCursos.



Captura 4 · Usuario autenticado en Mis inscripciones (operación MisInscripciones, solo devuelve las del usuario que llama).



Captura 5 · Emulator Suite: Authentication y SQL Connect corriendo localmente para el proyecto proyecto-pa-38252.

Parte D · Producción (L6, L7)

L6 · CSP. La política de la clase 1 tenía connect-src 'self', que bloquea las llamadas a Firebase. Se agregaron solo los orígenes que usa la app:

```
connect-src 'self'
  https://identitytoolkit.googleapis.com      (login y registro, Firebase Auth)
  https://securetoken.googleapis.com          (renovación del token de sesión)
  https://firebasedataconnect.googleapis.com (operaciones de SQL Connect)
```

L7 · Deploy. Se desplegó el servicio y se cargaron los datos iniciales en producción:

```
npx firebase login
npx firebase deploy --only dataconnect
npx firebase dataconnect:execute dataconnect/seed_data.gql
```

Preguntas de reflexión

1. ¿Por qué ninguna operación recibe el id del alumno? ¿Qué es IDOR?

No recibe el ID como variable porque el cliente es manipulable. IDOR ocurre cuando un atacante cambia un parámetro (ej. de alumnoId=1 a alumnoId=2) y accede a recursos ajenos. Usar auth.uid lo evita porque la identidad sale del token firmado por el servidor, imposible de falsificar.

2. ListCursos es PUBLIC y ListAlumnos terminó en NO_ACCESS. ¿Qué criterio se usó? ¿Cómo haría el panel de admin?

Se pasó a NO_ACCESS porque expone PII (nombres y emails) de forma irrestricta. El panel de admin debería acceder mediante un conector distinto configurado con Firebase IAM o usando el Admin SDK desde un entorno de confianza.

3. ¿Qué ventaja tiene la clave primaria compuesta frente a validar duplicados en el cliente?

Garantiza la unicidad a nivel de motor de base de datos. Si lo hiciéramos en el código cliente, dos requests simultáneos podrían generar una condición de carrera e insertar duplicados.

4. La configuración de Firebase es pública. ¿Por qué no compromete la base? ¿Qué capa faltaría?

La seguridad de Firebase no reside en esconder las credenciales (que de todas formas deben vivir en el cliente), sino en las reglas de autorización (el @auth de SQL Connect). Para frenar a un atacante que arme un script con tus claves, la capa que falta implementar es App Check (que certifica que el tráfico provenga genuinamente de tu app y no de un script de terminal).

5. Comparado con una API REST propia (Express + pg), ¿qué da SQL Connect y qué se pierde?

Se gana: no hay servidor que mantener ni escalar; la inyección SQL queda cerrada por diseño porque cada operación se compila a SQL parametrizado en el servidor; la autorización se declara junto a cada operación y es fácil de auditar; hay integración directa con Firebase Auth (auth.uid), migraciones asistidas y emuladores locales. **Se pierde:** flexibilidad para lógica de negocio compleja (requiere @check, @transaction o una Cloud Function aparte), control fino del SQL generado y portabilidad, por el acople a Google Cloud.

6. ¿Por qué no alcanzó con la CSP de la clase 1 y por qué conviene listar dominios exactos?

connect-src 'self' solo permite conexiones al mismo origen del sitio, y la app ahora habla con Firebase Auth y SQL Connect, que están en otros dominios: el navegador las bloqueaba (Refused to connect). Conviene listar dominios exactos porque un comodín como https://*.googleapis.com habilita cientos de APIs de Google; ante un XSS, un atacante podría exfiltrar datos hacia un recurso de Google bajo su control (por ejemplo, un bucket propio en Storage). Con la lista exacta se aplica mínimo privilegio a la red.